ELEC 6049 Digital system design techniques

Introduction to Digital Design

Department of Electrical and
Electronic Engineering

Acknowledgement

A lot of course materials here are adapted from the following sources:

- Prof. Zhongrui Wang, SUSTech
- ELEC3042 (HKU), Introduction to FPGA, by Dr. Hayden So
- CSE140 (UCSD), Introduction to Digital Logic Design, by Dr. Chung-Kuan Cheng
- EECS151 (Berkeley), Introduction to Digital Design and Integrated Circuits, by Dr. John Wawrzynek
- EECS303 (Northwestern), Advanced Digital Design, by Dr. Hai Zhou
- E2.1 (ICL), Introduction to Digital Circuits, by Dr. Peter Cheung
- Xilinx Workshop

Course Logistics

https://moodle.hku.hk/course/view.php?id=128800

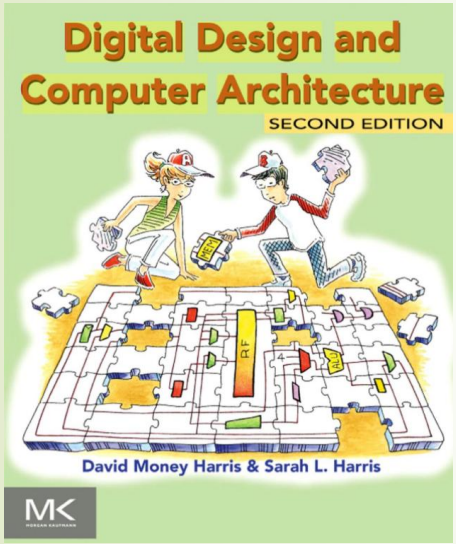
Lecture notes

- Assignments
- Projects guidelines

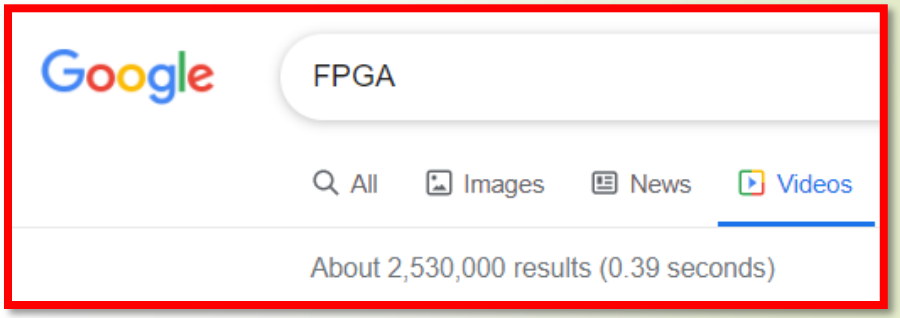
Meet our crew

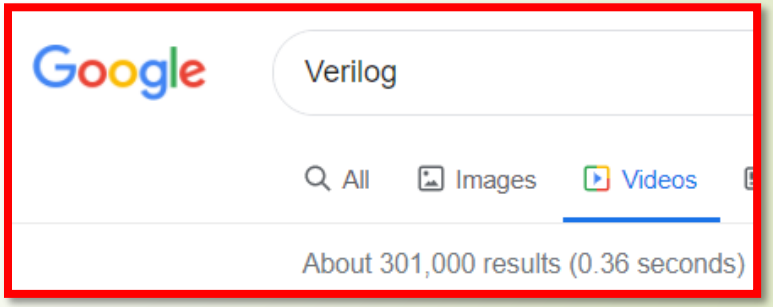
- Prof. Han Wang (hanwang6@hku.hk)
- Mr. Songqi Wang (songqi27@connect.hku.hk)
- Mr. Xinyuan Zhang (zhangxy8@connect.hku.hk)
- Mr. Mingzi Li (mzli@connect.hku.hk)

Want some textbooks to read?



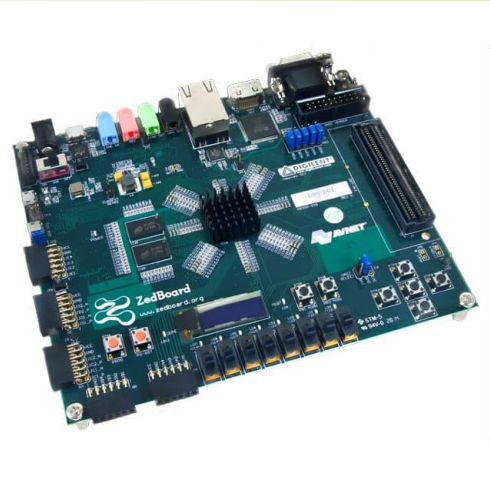
First >100 pages are available on Google Book. Check it out.





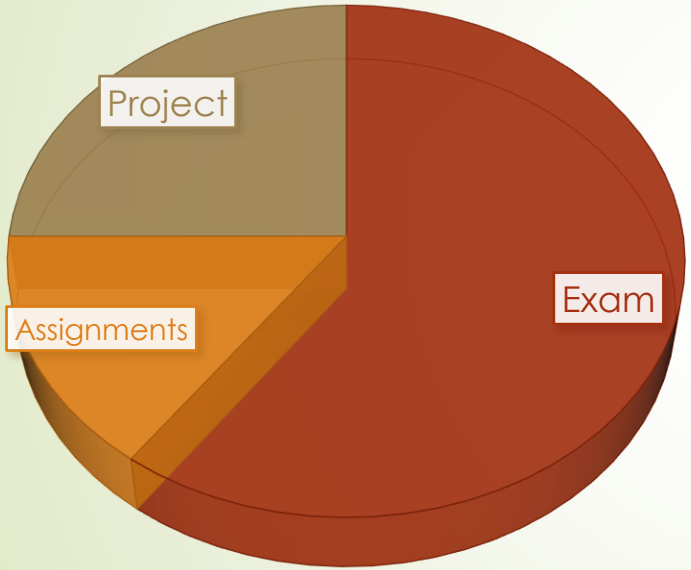
What are we going to do in this course?

- Lectures
  - First 6-7 lectures – Fundamentals of FPGA and Verilog
  - Last 4 classes – A small hands-on project
- 1 Final Exam



Course Timeline

Grading Breakdown



Exam	60%	1 final exam at the end of the course
Homework	15%	1 homework assignment
Project	25%	1 lab project

What is this course about?

```c
#include <stdio.h>
int main()
{
    printf("Hello, World!");
    return 0;
}
```


```
org 0x100
mov dx, msg
mov ah, 9
int 0x21
mov ah, 0x4c
int 0x21
msg db 'Hello, World!', 0x0d, 0x0a, '$'
```

High-level Language

Complier + Assembler + Linker

Instruction Set Architecture

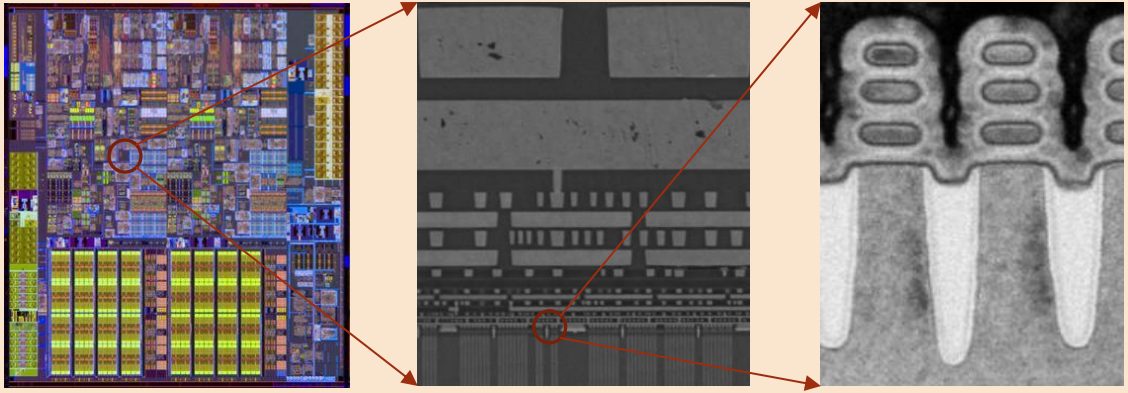
Hardware Descriptive Language

Logic gates

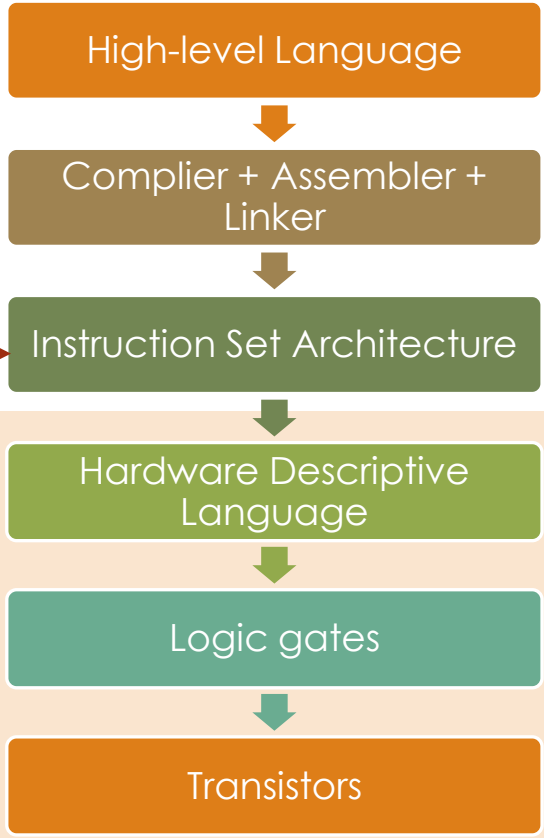
Transistors

What is this course about?

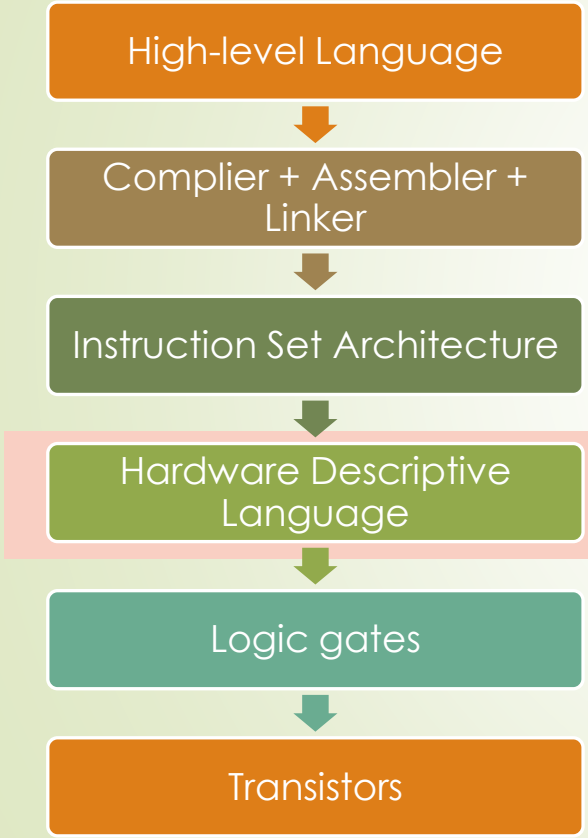
0x00…0x05, 0x80/0…0x81/0	Add
0x2F	Decimal adjust AL (*lower 8-bit of accumulator register*) after subtraction
0x3F	ASCII adjust AL after subtraction
…	…



Credit: Wikichip, Intel



What is this course about?



Lectures

- Introduction to Digital Design
- Basics of FPGA and Vivado
- Verilog Introduction
- Combinatorial Logic
- Sequential Logic
- Finite State Machine

Labs

- ZYNQ of Xilinx (ARM + FPGA)
- PYNQ environment
  - The Linux running on FPGA
- Jupyter Notebook on PYNQ
  - Run some python codes

Outline

What's the building block of digital circuit?
How do primitive logic gates work?

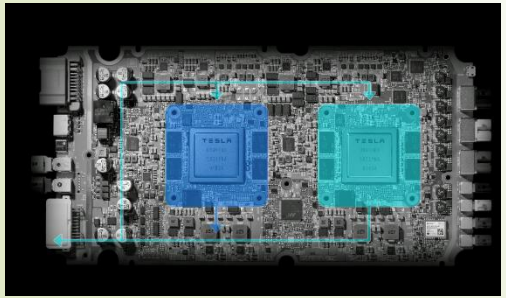
Brief history of Digital Design
Contemporary digital design

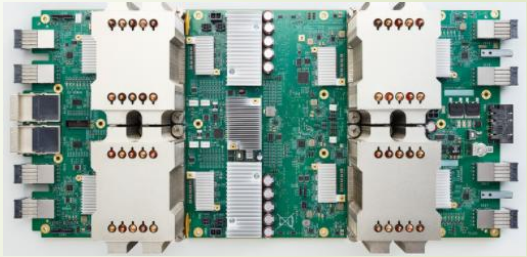
How to save "money" by making designs more versatile.

Digital Systems are Everywhere

Applications

Chips behind



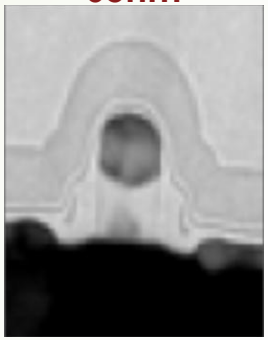


Credit: Tesla, Google, Qualcomm

Transistors : the building blocks

The First Transistor

65nm



45nm

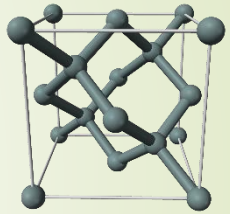
32nm

20nm

14nm

…

0.54nm



Si unit cell

First BJT transistor : by John Bardeen, Walter Brattain, William Shockley at Bell Labs.

First MOSFET : by Mohamed Atalla and Dawon Kahng in 1959 at Bell Labs.

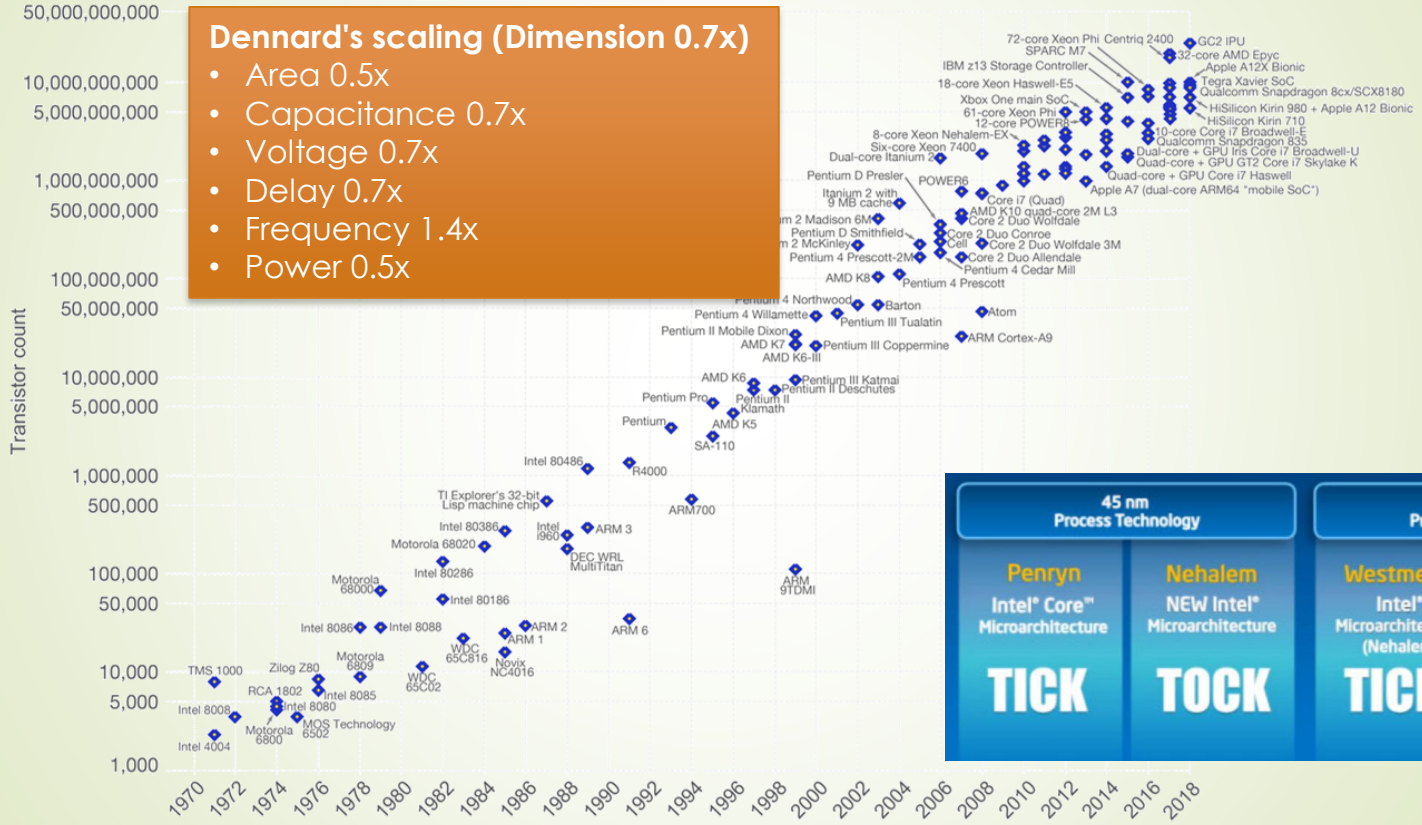
Followed by CMOS, FinFET…

Credit: Samsung, Wikipedia

Moore's law and Dennard's scaling

Dennard's scaling (Dimension 0.7x)

- Area 0.5x
- Capacitance 0.7x
- Voltage 0.7x
- Delay 0.7x
- Frequency 1.4x
- Power 0.5x

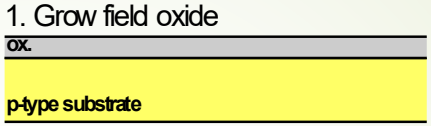


Credit: Wikipedia

Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

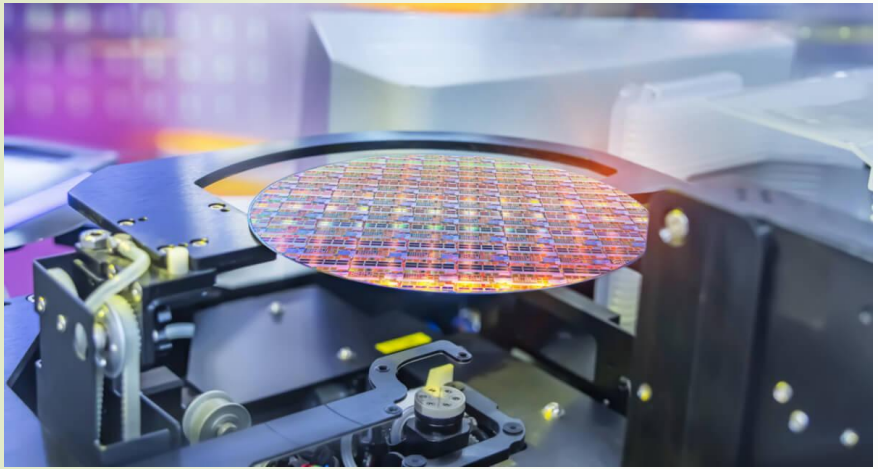
How to fabricate a CMOS?

- Like construction buildings! 3 primary techniques.
  - Deposition
  - Patterning
  - Etching



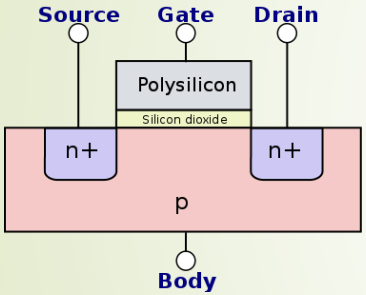
Credit: Wikipedia

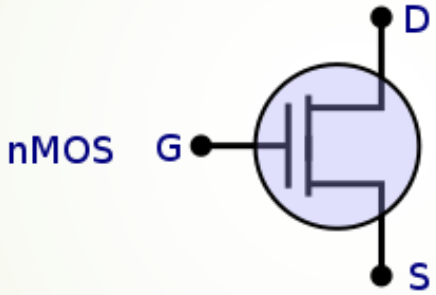
Making of a Chip

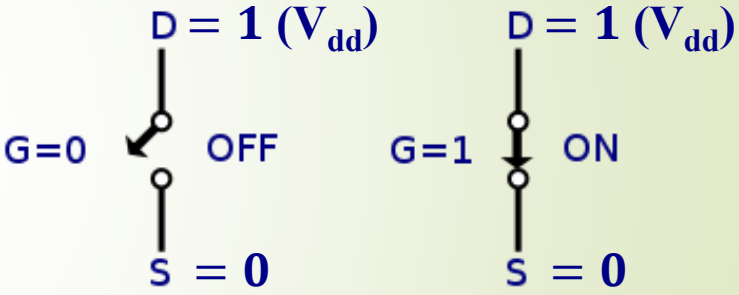


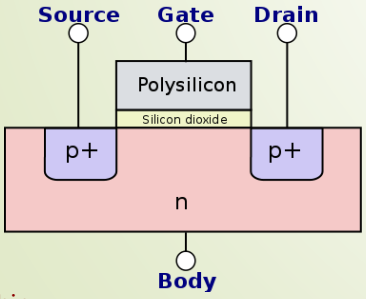
How do these transistors work?

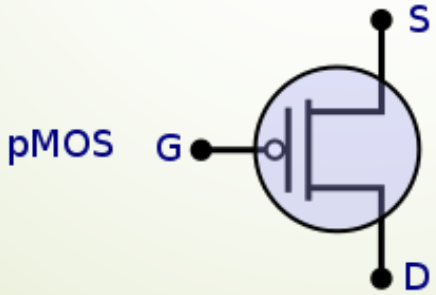
- MOS transistors are electrically controlled switches
  - Gate voltage-dependent source/drain-body junctions

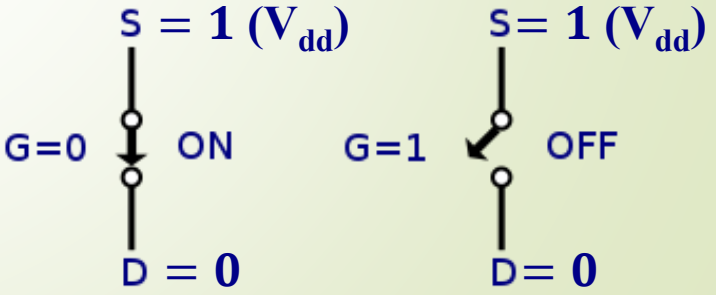












Credit: Wikichip

Recap: Primitive Boolean Logic Functions

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A \oplus B$	$\overline{A \oplus B}$
Symbol							

Truth Table

A	X
0	1
1	0

B	A	X
0	0	0
0	1	0
1	0	0
1	1	1

B	A	X
0	0	1
0	1	1
1	0	1
1	1	0

B	A	X
0	0	0
0	1	1
1	0	1
1	1	1

B	A	X
0	0	1
0	1	0
1	0	0
1	1	0

B	A	X
0	0	0
0	1	1
1	0	1
1	1	0

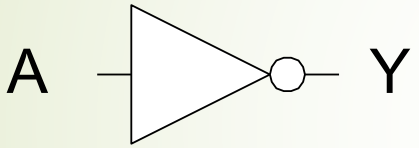
B	A	X
0	0	1
0	1	0
1	0	0
1	1	1

- **Universal**: can represent an arbitrary Boolean function
- **Black box**: with inputs/outputs and fixed mapping relation between input and output

Credit: Electronoobs

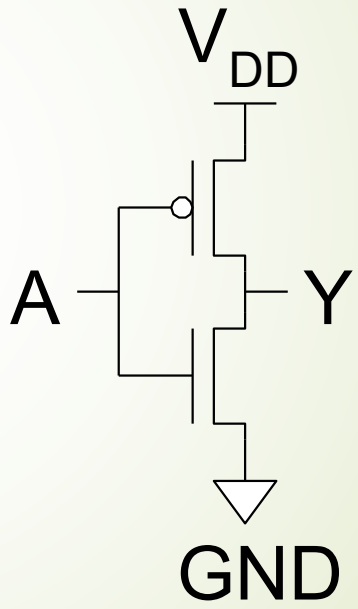
How do we realize these Boolean functions with CMOS?

Truth table of a NOT function



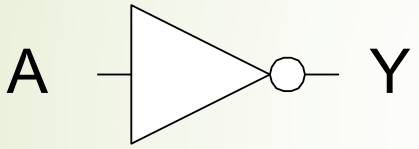
A	Y
0 (GND)	
1 (V_{DD})	

CMOS circuit of an invertor



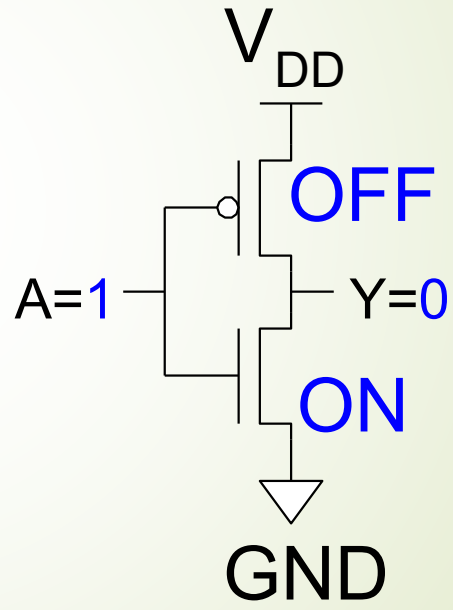
How do we realize these Boolean functions with CMOS?

Truth table of a NOT function



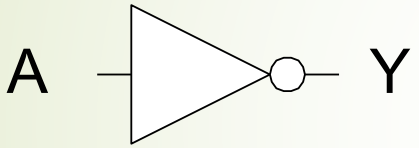
A	Y
0 (GND)	
1 (V_{DD})	0 (GND)

CMOS circuit of an invertor



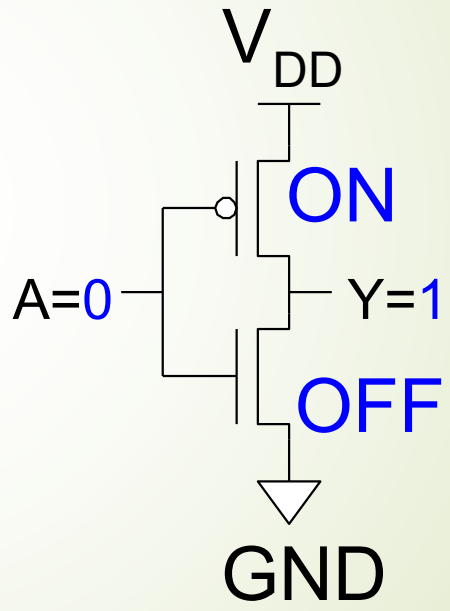
How do we realize these Boolean functions with CMOS?

Truth table of a NOT function



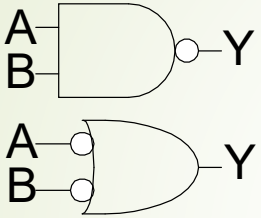
A	Y
0 (GND)	1 (V_{DD})
1 (V_{DD})	0 (GND)

CMOS circuit of an invertor



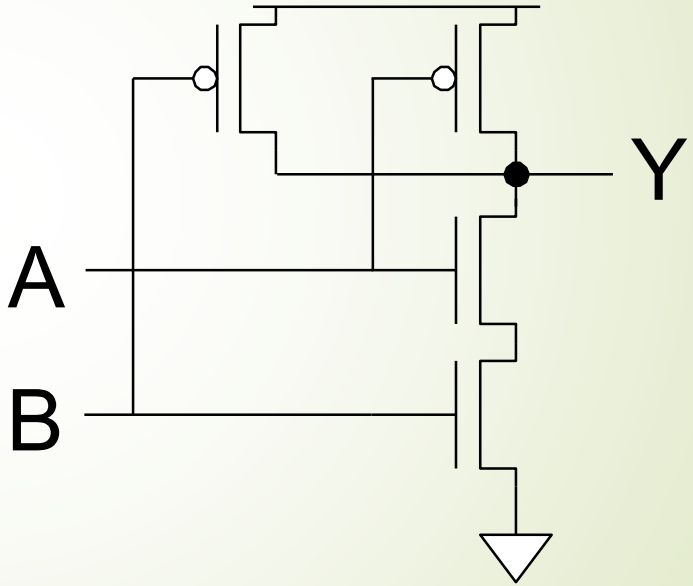
How do we realize these Boolean functions with CMOS?

Truth table of a NAND function



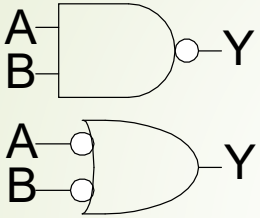
A	B	Y
0	0	
0	1	
1	0	
1	1	

CMOS circuit of a NAND gate



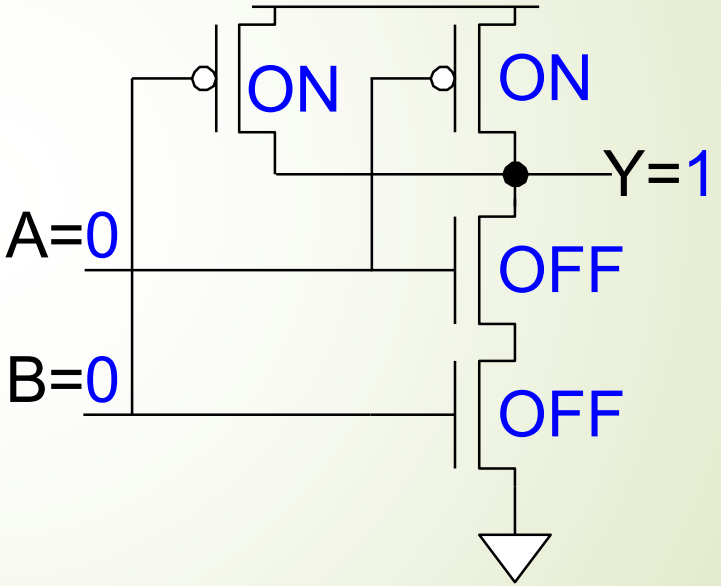
How do we realize these Boolean functions with CMOS?

Truth table of a NAND function



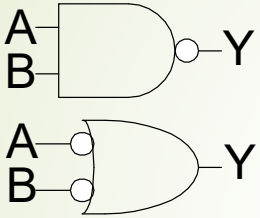
A	B	Y
0	0	1
0	1	
1	0	
1	1	

CMOS circuit of a NAND gate



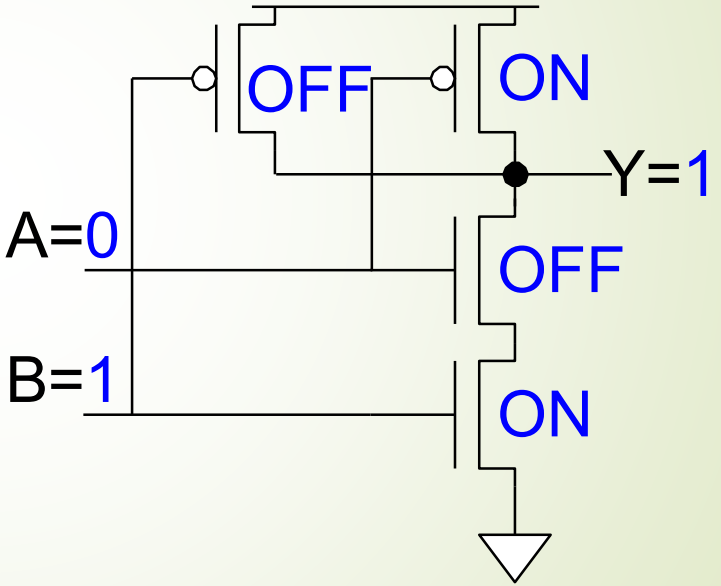
How do we realize these Boolean functions with CMOS?

Truth table of a NAND function



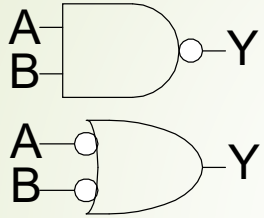
A	B	Y
0	0	1
0	1	1
1	0	
1	1	

CMOS circuit of a NAND gate



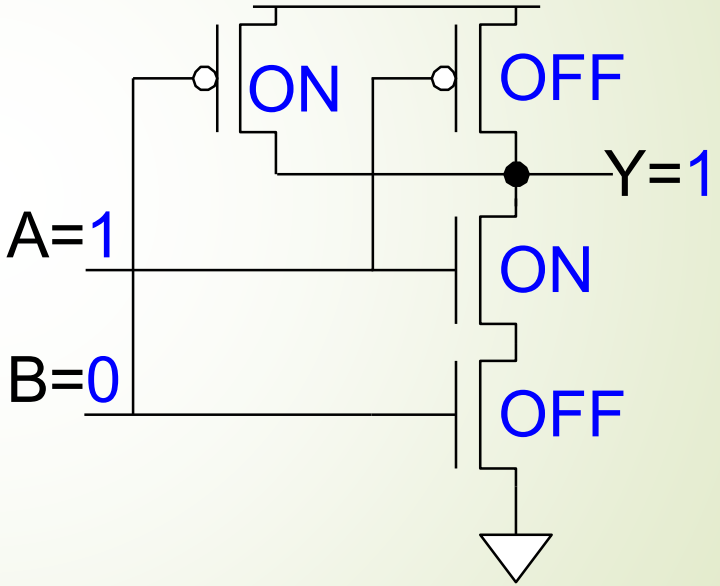
How do we realize these Boolean functions with CMOS?

Truth table of a NAND function



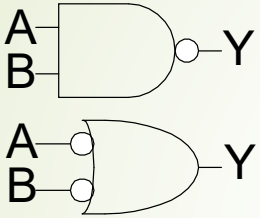
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	

CMOS circuit of a NAND gate



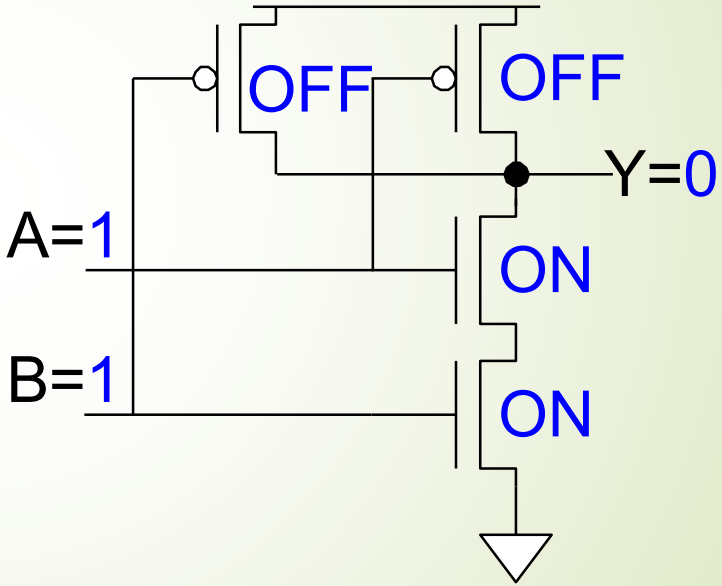
How do we realize these Boolean functions with CMOS?

Truth table of a NAND function



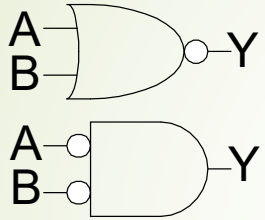
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

CMOS circuit of a NAND gate



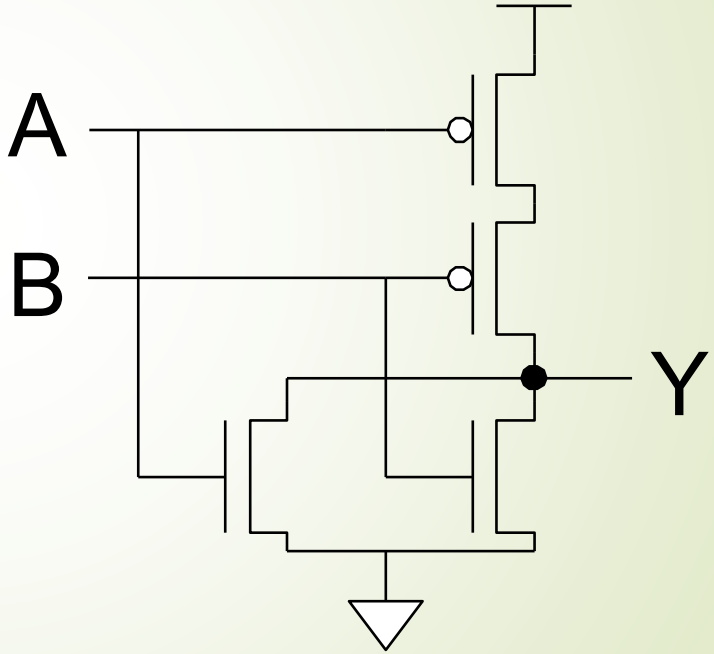
How do we realize these Boolean functions with CMOS?

Truth table of a NOR function



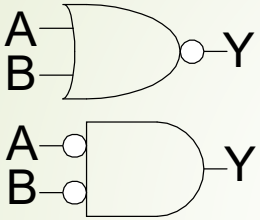
A	B	Y
0	0	
0	1	
1	0	
1	1	

CMOS circuit of a NOR gate



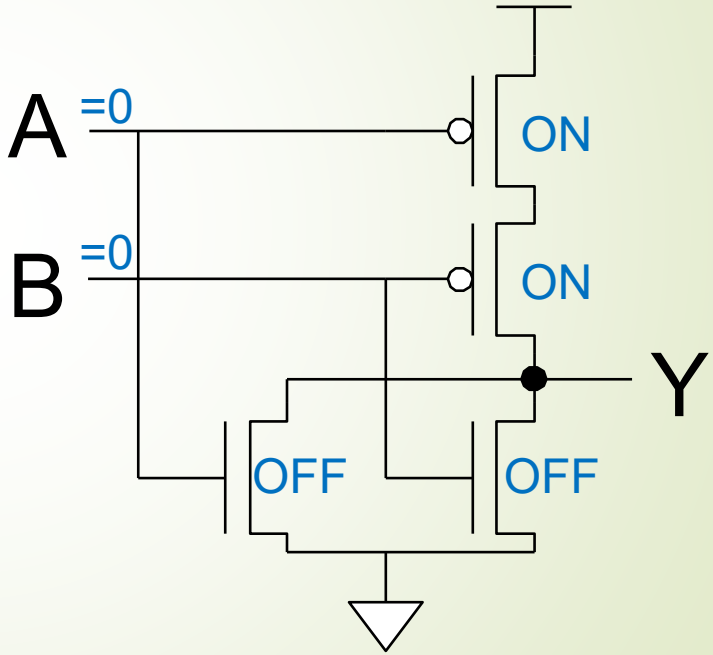
How do we realize these Boolean functions with CMOS?

Truth table of a NOR function



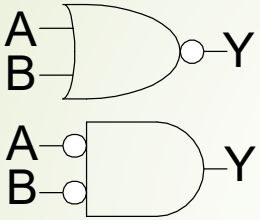
A	B	Y
0	0	1
0	1	
1	0	
1	1	

CMOS circuit of a NOR gate



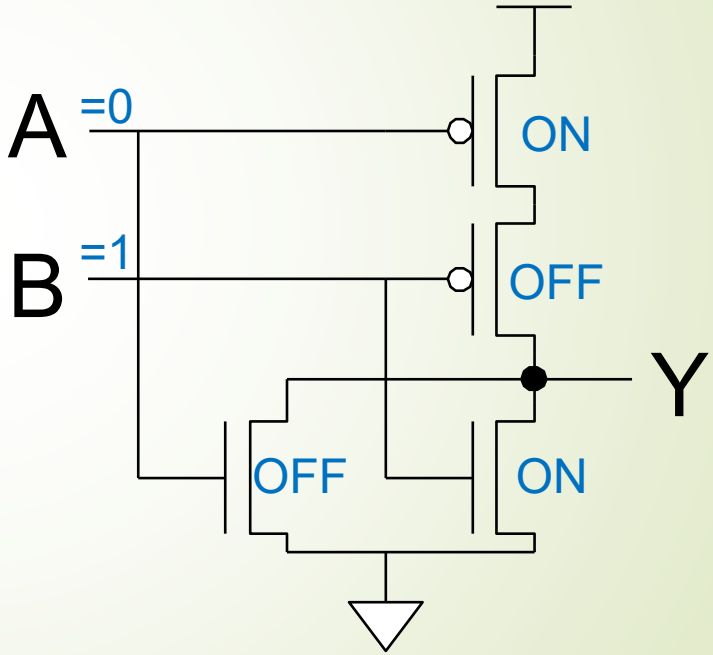
How do we realize these Boolean functions with CMOS?

Truth table of a NOR function



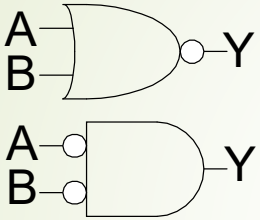
A	B	Y
0	0	1
0	1	0
1	0	
1	1	

CMOS circuit of a NOR gate



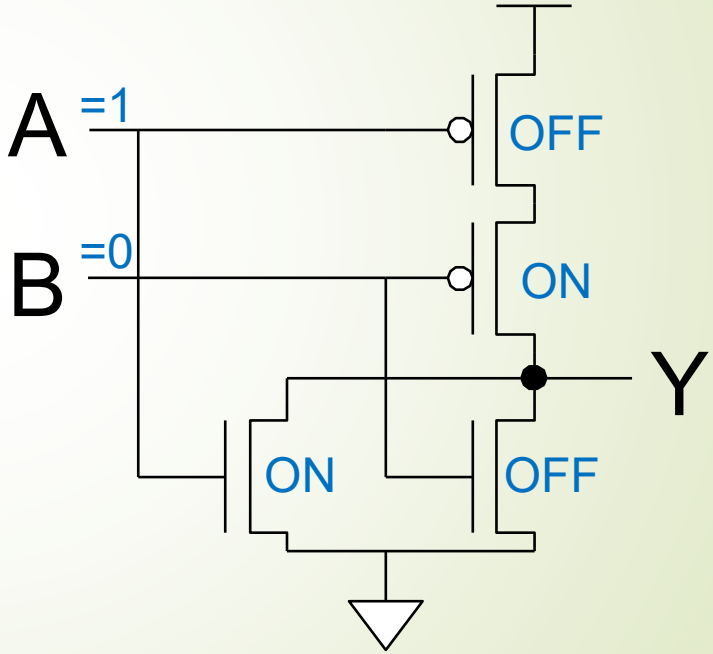
How do we realize these Boolean functions with CMOS?

Truth table of a NOR function



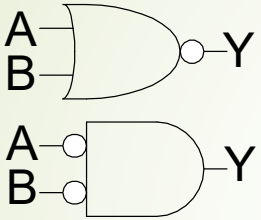
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	

CMOS circuit of a NOR gate



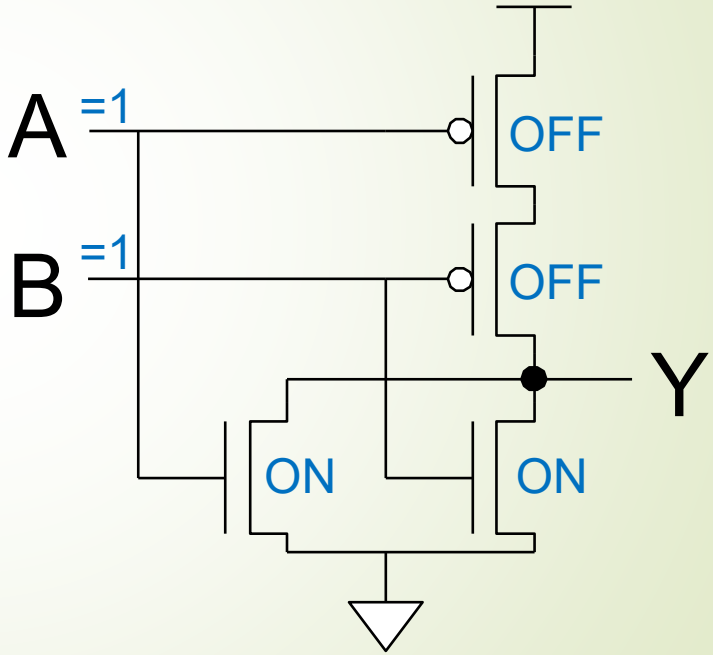
How do we realize these Boolean functions with CMOS?

Truth table of a NOR function



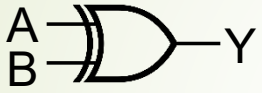
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

CMOS circuit of a NOR gate



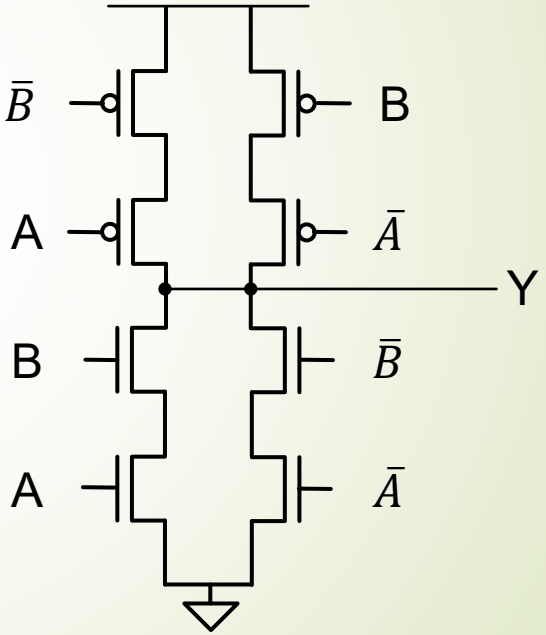
How do we realize these Boolean functions with CMOS?

Truth table of a XOR function



A	B	Y
0	0	
0	1	
1	0	
1	1	

CMOS circuit of a XOR gate

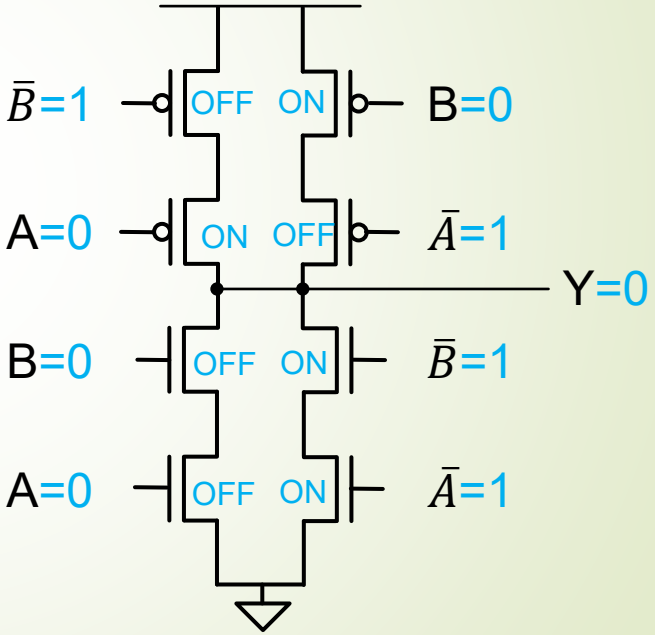


How do we realize these Boolean functions with CMOS?

Truth table of a XOR function

A	B	Y
0	0	0

CMOS circuit of a XOR gate



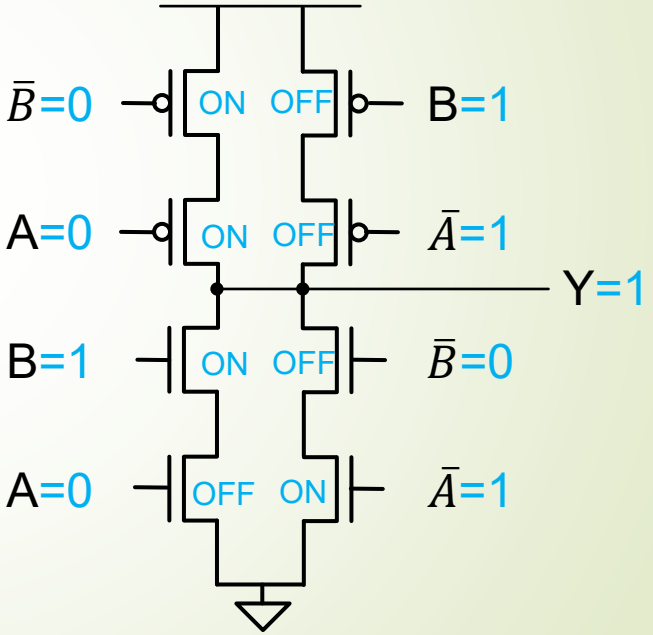
How do we realize these Boolean functions with CMOS?

Truth table of a XOR function

A
B $\bar{B}=0$ Y

A	B	Y
0	0	0
0	1	1

CMOS circuit of a XOR gate

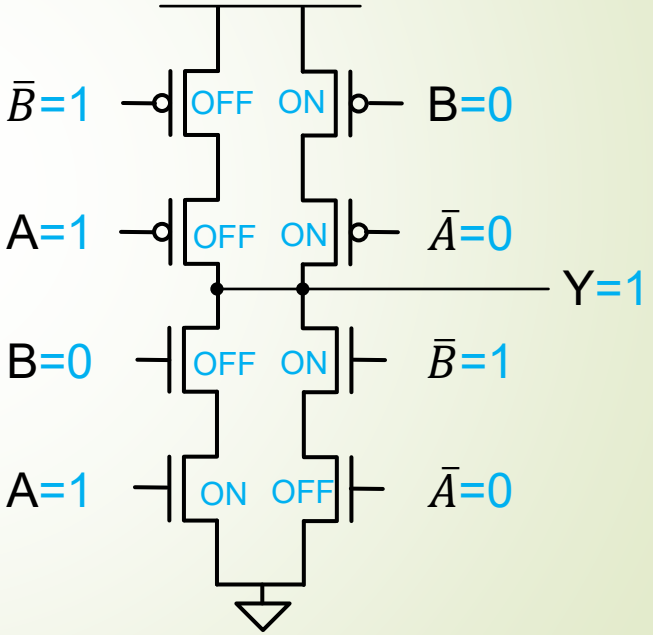


How do we realize these Boolean functions with CMOS?

Truth table of a XOR function

A	B	Y
0	0	0
0	1	1
1	0	1

CMOS circuit of a XOR gate



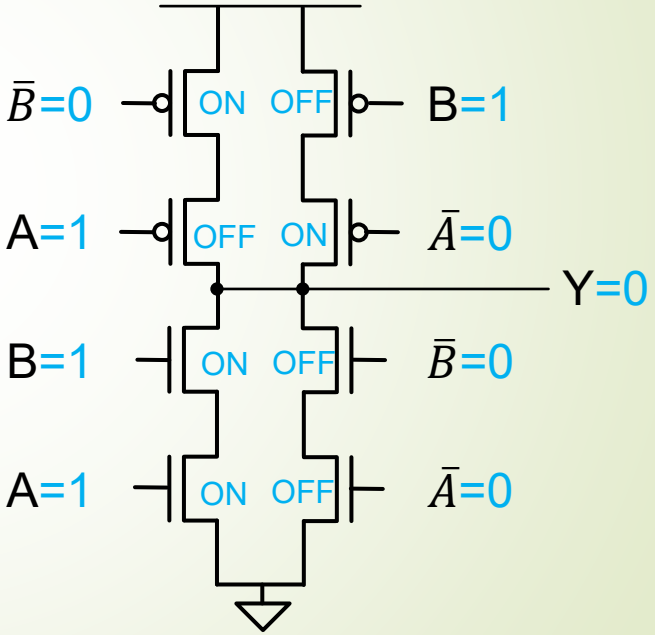
How do we realize these Boolean functions with CMOS?

Truth table of a XOR function

A B $\bar{A}$=0 Y

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

CMOS circuit of a XOR gate

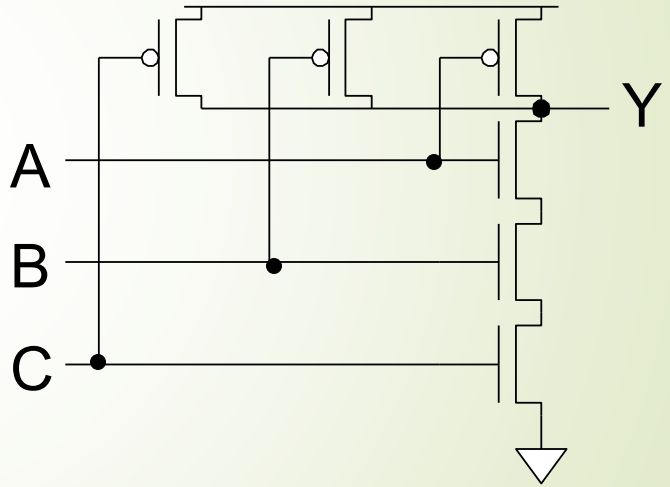


How about a 3-input NAND Gate?

Truth table of a 3-input NAND function

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

CMOS circuit of a 3-input NAND gate



- Y is pulled down (0) if ALL inputs are 1
- Y is pulled up (1) if ANY input is 0

Outline

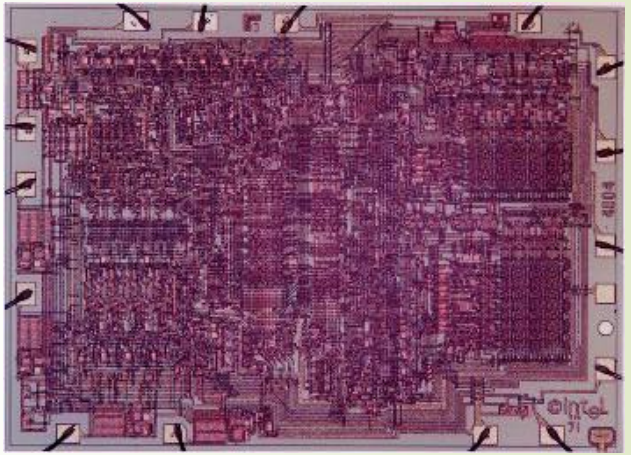
What's the building block of digital circuit?
How do primitive logic gates work?

Brief history of Digital Design
Contemporary digital design

How to save "money" by making designs more versatile.

Digital Design in the 70's and early 80's

- Layout : hand drawings.
- Photo masks : rubylith mask (hand cutting)

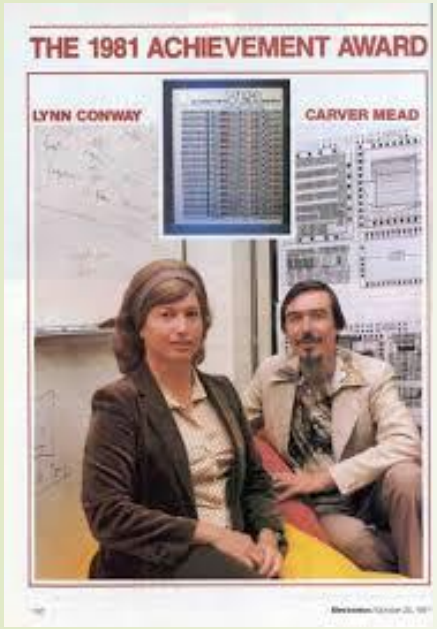


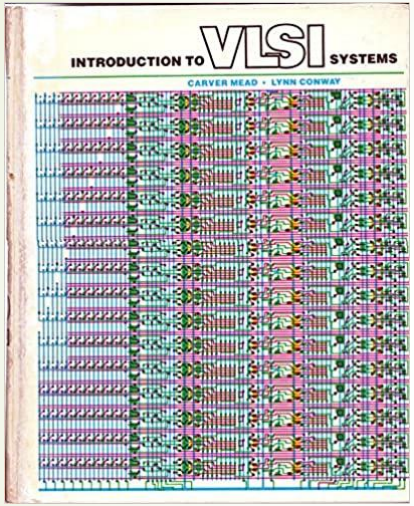
Example : Intel 4004 CPU (1971)

- 2300 transistors
- ~60,000 calculations/s

Credit: Jim Wiley, Intel

Mead & Conway revolution and Modern Design





EDA Circuit Design

NVIDIA, AMD, QUALCOMM®

Mask designs

Process design kit

Chip Fabrication

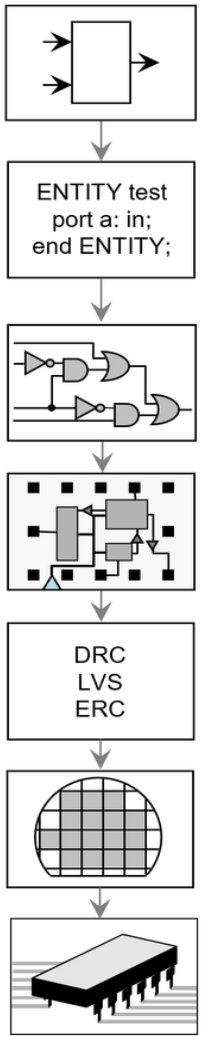
tsmc, SAMSUNG 삼성전자, GLOBALFOUNDRIES, UMC

Credit: Electronics

Conventional Design Flow

cadence

SYNOPSYS®



System Specification

Architectural Design

Functional Design and Logic Design

Circuit Design

Physical Design

Physical Verification and Signoff

Layout Post Processing

Fabrication

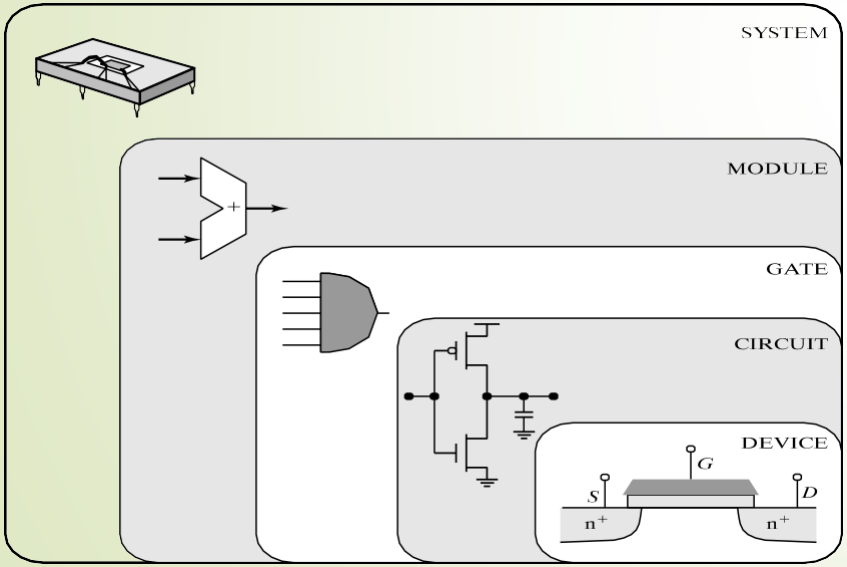
Packaging and Testing

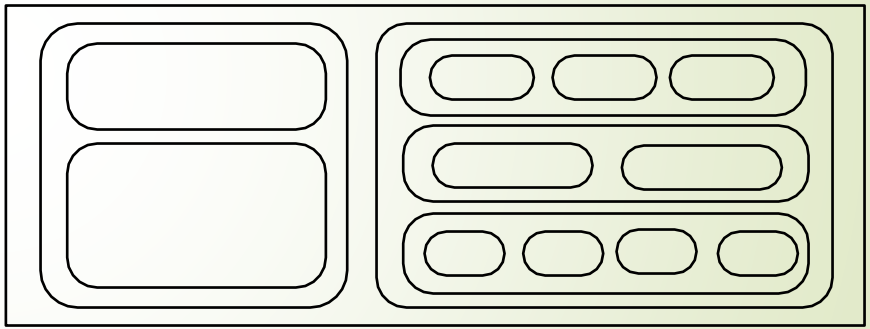
- Partitioning
- Floorplanning
- Placement
- Clock Tree Synthesis
- Signal Routing
- Timing Closure

Credit: Wikipedia

Advantage of design hierarchy : abstraction

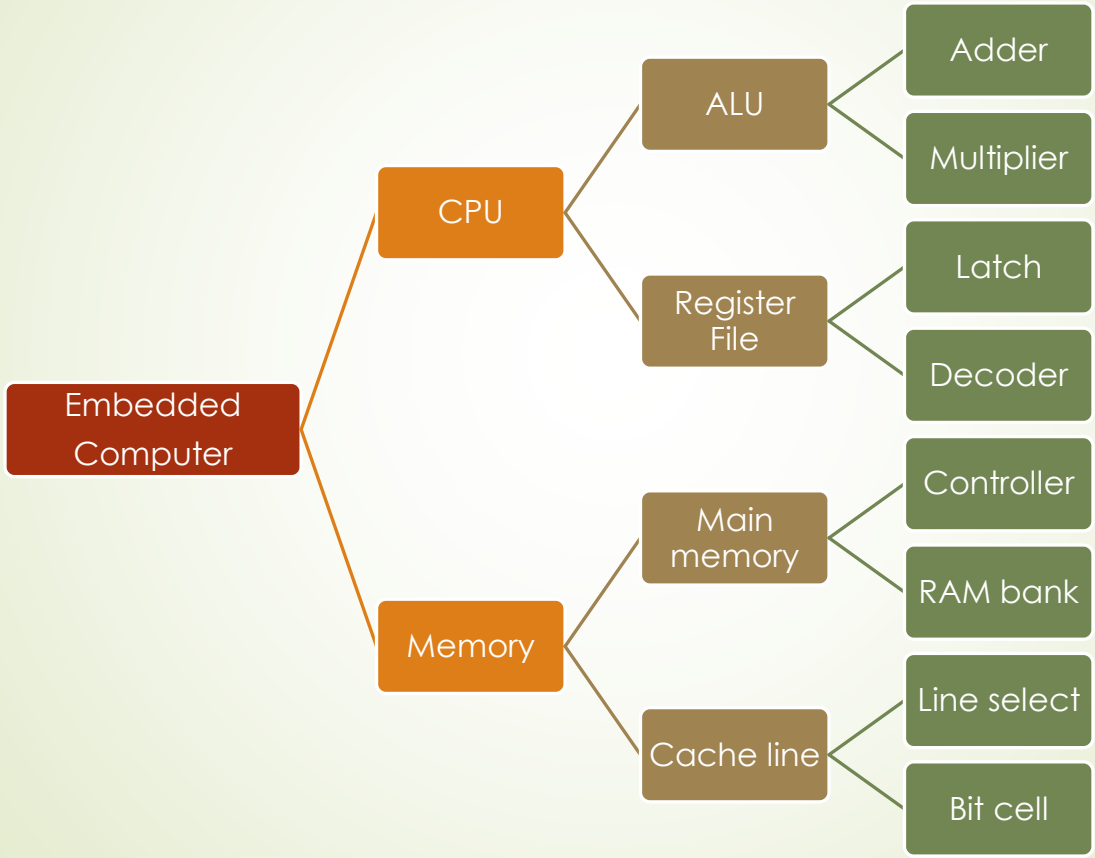
- Design Abstraction
  - Hide details and reduce number of things to handle at any time
- Modular design
  - Divide and conquer
  - Simplifies implementation and debugging





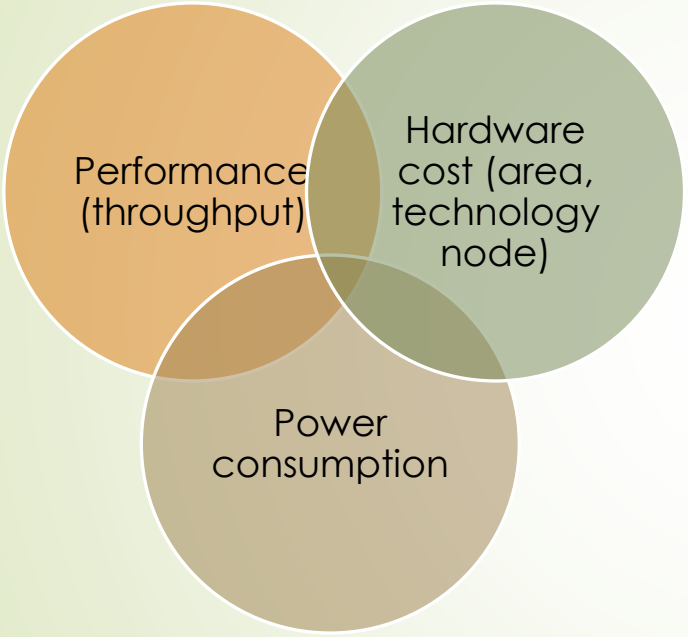
Design methodology : Where to begin?

Top module



Bottom modules

Digital design : An optimization problem



- An optimization problem
- For example, utility function $util = T - P \times 0.1 - \mathrm{A} \times 0.5$, T, P, A are through

Var	Definition	Remarks
T	Throughput	Function of the design
P	Power	
A	Area	

- Optimization problems within every level in the system design

Outline

What's the building block of digital circuit?
How do primitive logic gates work?

Brief history of Digital Design
Contemporary digital design

How to save "money" by making designs more versatile.

How to save “money”

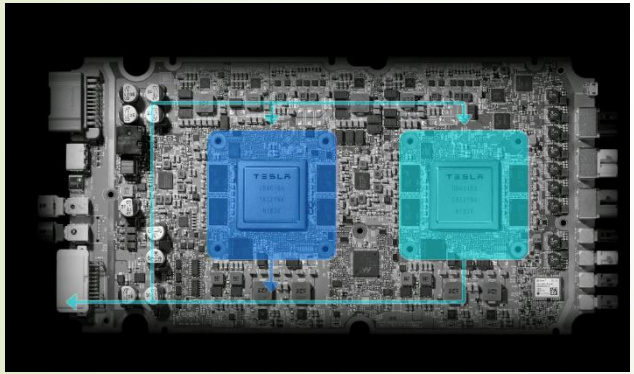
More General

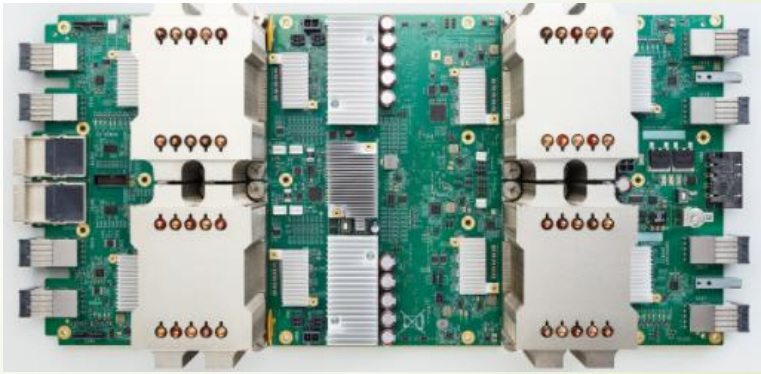
Fully-customized	Each transistor has been optimized towards the goal.
Standard-cell	Cells (e.g. gates, FFs) are pre-designed.
Gate-array (structured ASIC)	Partially prefabricated wafers customized with metal layers or vias.
FPGA	Prefabricated chips customized with loadable latches or fuses.
Domain Specific Processor	Special instruction set interpreters (ex: DSP, NP, GPU).
Microprocessor	Instruction set interpreter customized through soft ware.

More Efficient

Fully customized chip

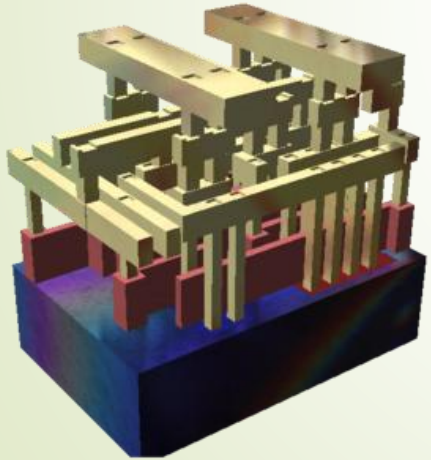
- Like you specify where to place each brick in building a house.
- Fully optimized (and thus highly efficient) for the targeting application.
- Very costly to develop.





Standard-Cell

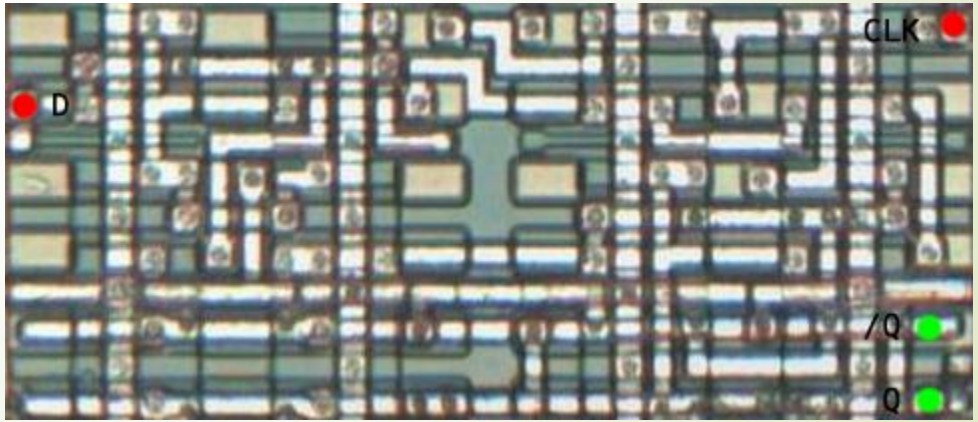
- Like building a house with pre-fabricated panels/roofs.
- Arrays of small function blocks (gates, Flip-flops) automatically placed and routed.



Credit: Wiki, Chim, W.M.

Gate Array

- Like partition an office building floor with walls to form functional rooms
- Partially prefabricated wafers customized with metal layers or vias.
- The master slice can be reused by different customers without touching the transistors, but making customized wiring.



Credit: Ebay, Fujitsu